

Отчет по лабораторной работе №2

Структуры данных

Еремина Оксана Андреевна

В данном отчёте представлены результаты выполнения лабораторной работы по теме ...

Описаны использованные методы, полученные экспериментальные данные и их анализ.

Сделаны выводы о достижении поставленной цели.

Содержание

1	Введение	6
1.1	Актуальность	6
1.2	Цель работы	6
1.3	Задачи	6
1.4	Объект и предмет исследования	7
1.5	Методы исследования	7
2	Теоретическая часть	8
3	Материалы и методы	9
4	Выполнение лабораторной работы	10
5	Результаты	15
6	Обсуждение результатов	16
7	Заключение	17
	Список литературы	18

Список иллюстраций

4.1	Задание 1. Множества	10
4.2	Задание 2. Разность и принадлежность	11
4.3	Задание 3 Множества	11
4.4	Задание 4. squares	12
4.5	Задание 5	12
4.6	Задание 6	13
4.7	Задание 7	13
4.8	Задание 8	13
4.9	Задание 8	13
4.10	Задание 8	14

Список таблиц

1 Введение

1.1 Актуальность

Изучение структур данных является фундаментальной частью подготовки специалистов в области информационных технологий. Владение различными структурами данных и операциями над ними позволяет эффективно решать широкий круг прикладных задач, от обработки данных до реализации алгоритмов.

1.2 Цель работы

Основная цель работы — изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач [bezanson_julia_2017].

1.3 Задачи

Для достижения цели необходимо выполнить следующие шаги:

1. Ознакомиться с теоретическими сведениями о кортежах, словарях, множествах и массивах в Julia.
2. Повторить примеры из раздела 2.2 методических указаний.
3. Выполнить задания для самостоятельной работы (раздел 2.4).
4. Оформить результаты в виде отчета с листингами кода и скриншотами.

1.4 Объект и предмет исследования

Объектом исследования являются структуры данных языка программирования Julia. Предметом исследования — синтаксис, методы создания и операции над кортежами, множествами и массивами.

1.5 Методы исследования

В работе использовались практические методы программирования, анализ документации языка Julia [`dox_julia_sets`; `dox_julia_tuples`; `dox_julia_arrays`], а также экспериментальная проверка работы кода в среде JupyterLab.

2 Теоретическая часть

В языке Julia существует несколько встроенных структур данных. В данной работе рассматриваются кортежи, словари, множества и массивы.

Кортежи (Tuple) — это неизменяемая индексируемая последовательность элементов. Они полезны для группировки значений разных типов [[dox_julia_tuples](#)].

Словари (Dict) — неупорядоченный набор пар «ключ-значение». Позволяют быстро находить значения по ключу.

Множества (Set) — неупорядоченная совокупность уникальных элементов. Поддерживают математические операции: объединение, пересечение, разность [[dox_julia_sets](#)].

Массивы (Array) — коллекция упорядоченных элементов, размещённая в многомерной сетке. Векторы и матрицы являются частными случаями массивов. Julia поддерживает мощные средства для работы с массивами, включая генераторы списков [[dox_julia_arrays](#)].

3 Материалы и методы

Работа выполнялась в среде JupyterLab с использованием языка программирования Julia (версия 1.12). Для выполнения задания 5 был подключен пакет `Primes` [`primes_julia_pkg`]. Все вычисления проводились на локальной машине.

4 Выполнение лабораторной работы

4.0.1 Задание 1. Множества

Даны множества: $A = \{0, 3, 4, 9\}$, $B = \{1, 3, 4, 7\}$, $C = \{0, 1, 2, 4, 7, 8, 9\}$. Необходимо найти $P = A \cap B \cup A \cap C \cup B \cap C$.

Код для выполнения задания и результат его работы представлены на рис. 4.1.

```
(a = 2, b = 3)

julia> println("length(x2) = ", length(x2))
length(x2) = 3

julia> println("x2[1], x2[2], x2[3] = ", x2[1], ", ", x2[2], ", ", x2[3])
x2[1], x2[2], x2[3] = 1, 2.0, tmp

julia> c = x1[2] + x1[3]
5

julia> println("c = x1[2] + x1[3] = ", c)
c = x1[2] + x1[3] = 5

julia> println("x3.a = ", x3.a, ", x3.b = ", x3.b, ", x3[2] = ", x3[2])
x3.a = 2, x3.b = 3, x3[2] = 3

julia> println("in(\"tmp\", x2) = ", in("tmp", x2))
in("tmp", x2) = true
```

Рисунок 4.1: Задание 1. Множества

4.0.2 Задание 2. Свои примеры с разными типами

Были созданы множества целых чисел, строк и смешанных типов. Выполнены операции объединения и пересечения (рис. ?@fig-003).

```

( "807-5509", "333-5544" )

julia> println("После удаления: ", collect(pairs(phonebook)))
После удаления: Pair{String, Any}["Сидоров П.С." => "555-3344", "Бухгалтер"
"555-2368"]

julia> a = Dict{"foo" => 0.0, "bar" => 42.0}
Dict{String, Float64} with 2 entries:
  "foo" => 0.0
  "bar" => 42.0

julia> b = Dict{"baz" => 17, "bar" => 13.0}
Dict{String, Real} with 2 entries:
  "bar" => 13.0
  "baz" => 17

julia> println("merge(a, b) = ", merge(a, b))
merge(a, b) = Dict{String, Real}{"foo" => 0.0, "bar" => 13.0, "baz" => 17

julia> println("merge(b, a) = ", merge(b, a))
merge(b, a) = Dict{String, Real}{"foo" => 0.0, "bar" => 42.0, "baz" => 17

```

Рисунок 4.2: Задание 2. Разность и принадлежность

4.0.3 Задание 3. Множества

Созданы кортежи с последовательностями чисел от 1 до N (N=25): возрастающие, убывающие, симметричные, а также произвольный кортеж (рис. 4.3).

```

julia> println("E = setdiff(S3,S1) = ", E)
E = setdiff(S3,S1) = Set{[3]}

julia> println("issubset(S1,S4) = ", issubset(S1,S4))
issubset(S1,S4) = true

julia> push!(S4, 99)
Set{Int64} with 4 elements:
 3
 2
 1
 99

julia> println("После push!(S4, 99): ", S4)
После push!(S4, 99): Set{[3, 2, 1, 99]}

julia> pop!(S4)
3

julia> println("После pop!(S4): ", S4)
После pop!(S4): Set{[2, 1, 99]}

```

Рисунок 4.3: Задание 3 Множества

4.0.4 Задание 4. Массивы

Создан массив квадратов чисел от 1 до 100, выведены первые и последние 10 элементов (рис. 4.4).

```

2.23606797749979
2.449489742783178
2.6457513110645907
2.8284271247461903
3.0
3.1622776601683795

julia> println("roots = ", roots)
roots = [1.0, 1.4142135623730951, 1.7320508075688772, 2.0, 2.236067977499
49489742783178, 2.6457513110645907, 2.8284271247461903, 3.0, 3.1622776601

julia> odd_cubes = [3*x^2 for x in 1:2:9]
5-element Vector{Int64}:
 3
 27
 75
147
243

julia> println("odd_cubes = ", odd_cubes)
odd_cubes = [3, 27, 75, 147, 243]

```

Рисунок 4.4: Задание 4. squares

4.0.5 Задание 5. Операции над векторами x и y

```

julia> y = [500, 720, 300, 900, 450, 610, 800, 350, 950, 400,
            700, 480, 250, 980, 150, 760, 420, 880, 640, 200]
20-element Vector{Int64}:
500
720
300
900
450
610
800
350
950
400
700
480
250
980
150
760
420
880
640
200

```

Рисунок 4.5: Задание 5

4.0.6 Задание 6. Squares

```
julia> println("squares = ", squares)
squares = [1, 9, 25, 49, 81, 121, 169, 225, 289, 361, 441, 529, 625, 729, 841, 961, 1089, 1225, 1369, 1521, 1681, 1849, 2025, 2209, 2401, 2601, 2809, 3025, 3249, 3481, 3721, 3969, 4225, 4489, 4761, 5041, 5329, 5625, 5929, 6241, 6561, 6889, 7225, 7569, 7921, 8281, 8649, 9025, 9409, 9801]

julia> println("Длина: ", length(squares))
Длина: 50

julia>
```

Рисунок 4.6: Задание 6

4.0.7 Задание 7. Primes

```
@ REPL[134]:1

julia>

julia> s1 = sum(i^3 + 4*i^2 for i in 10:100)
26852735

julia> println("6.1) s1 = ", s1)
6.1) s1 = 26852735

julia>
```

Рисунок 4.7: Задание 7

4.0.8 Задание 8.1 Вычисление выражений

```
julia> M = 25
25

julia> s2 = sum(2 + 3/i for i in 1:M)
61.44787453326052

julia> println("6.2) s2 = ", s2)
6.2) s2 = 61.44787453326052

julia>
```

Рисунок 4.8: Задание 8

4.0.9 Задание 8.2 Вычисление выражений

```
julia> s2 = sum(2 + 3/i for i in 1:25)
61.44787453326052

julia> println(s2)
61.44787453326052

julia> # 61.447874...

julia>
```

Рисунок 4.9: Задание 8

4.0.10 Задание 8.3 Вычисление выражений

```
julia> function term3(n)
    p = 1.0
    for k in 1:n
        p *= (2k) / (2k + 1)
    end
    return p
end
term3 (generic function with 1 method)

julia> s3 = 1.0 + sum(term3(n) for n in 1:19)
6.976346137897619

julia> println("6.3) s3 = ", s3)
6.3) s3 = 6.976346137897619

julia>
```

Рисунок 4.10: Задание 8

5 Результаты

В ходе выполнения лабораторной работы были получены следующие результаты:

- Изучены синтаксис и операции для кортежей, словарей, множеств и массивов в Julia.
- Успешно выполнены все 14 пунктов заданий для самостоятельной работы.
- Получены числовые результаты для задач 3.9, 3.10, 3.14, 6.1, 6.2, 6.3.
- Сформированы векторы и массивы в соответствии с заданиями.

6 Обсуждение результатов

Результаты выполнения заданий полностью соответствуют ожидаемым. Использование генераторов списков и встроенных функций Julia (таких как `union`, `intersect`, `setdiff`, `findall`, `sortperm`) позволило компактно и эффективно решить поставленные задачи [bezanson_julia_2017]. Трудностей при выполнении не возникло, так как синтаксис языка интуитивно понятен, а документация доступна.

7 Заключение

В ходе лабораторной работы были изучены основные структуры данных в Julia: кортежи, словари, множества и массивы. Были освоены методы их создания и операции над ними. Все задания для самостоятельной работы выполнены в полном объеме. Цель работы достигнута.

Список литературы